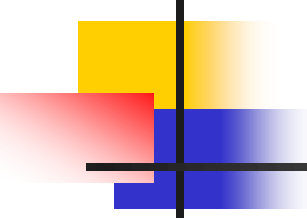


ECSE 4320/6320 Advanced Computer Systems

Introduction

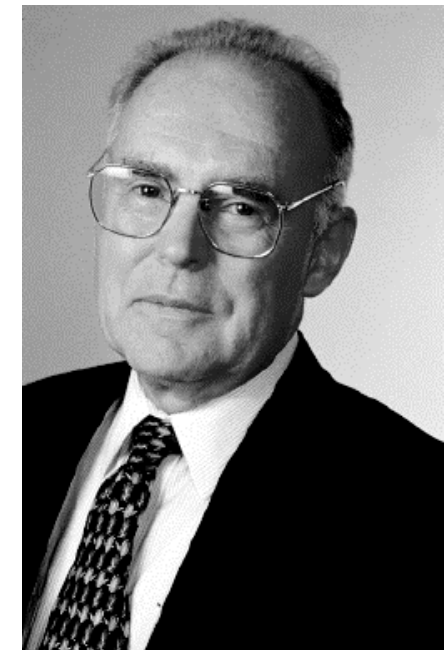
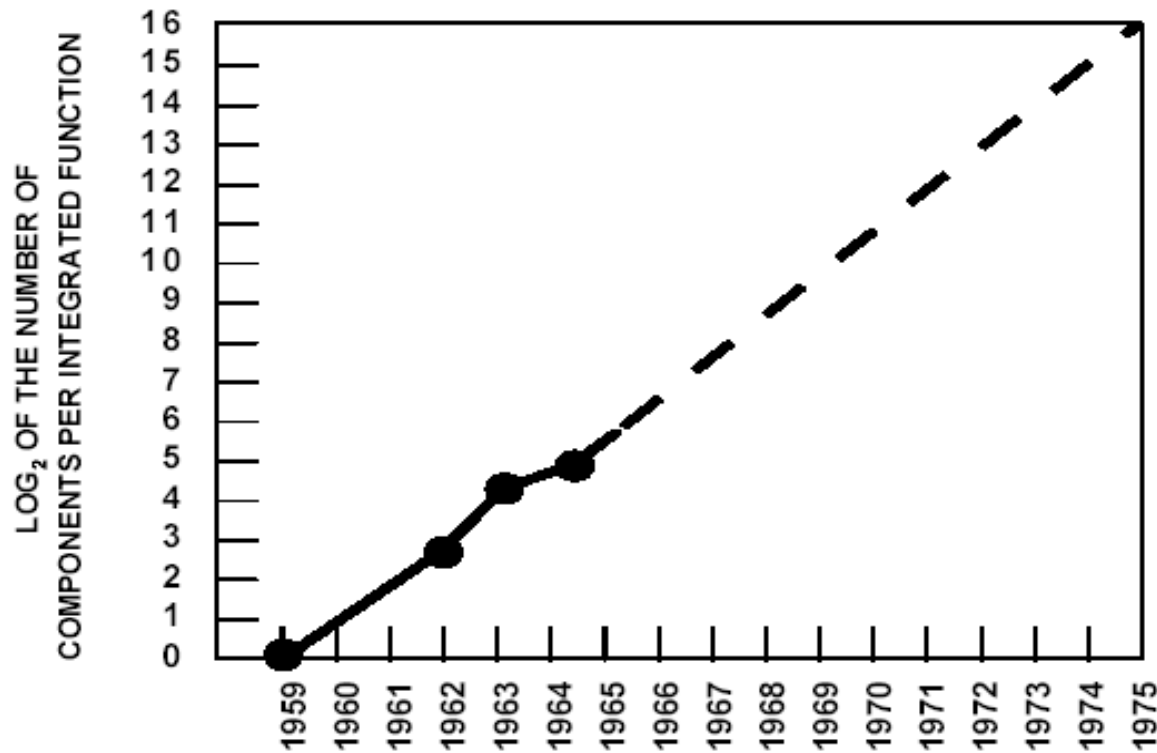


General Information

- ☐ **Why** take this course?
- ☐ **What** is this course all about?
- ☐ **Who** is teaching this course?
- ☐ **How** will this course work?

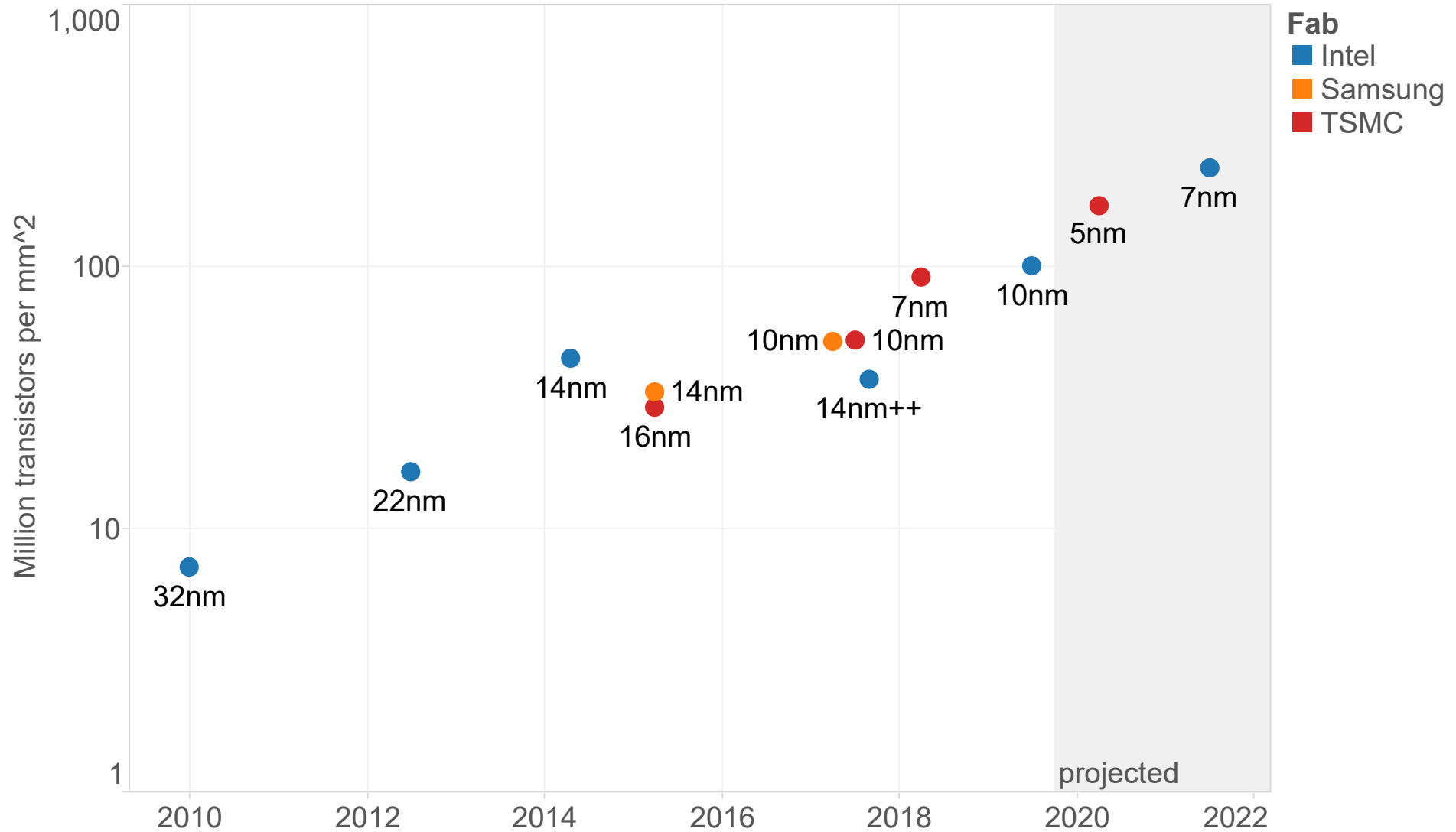
Why? & What?

In 1965, Gordon Moore (co-founder of Intel) made a prediction that semiconductor technology will double its effectiveness every 18 months



Electronics, April 19, 1965.

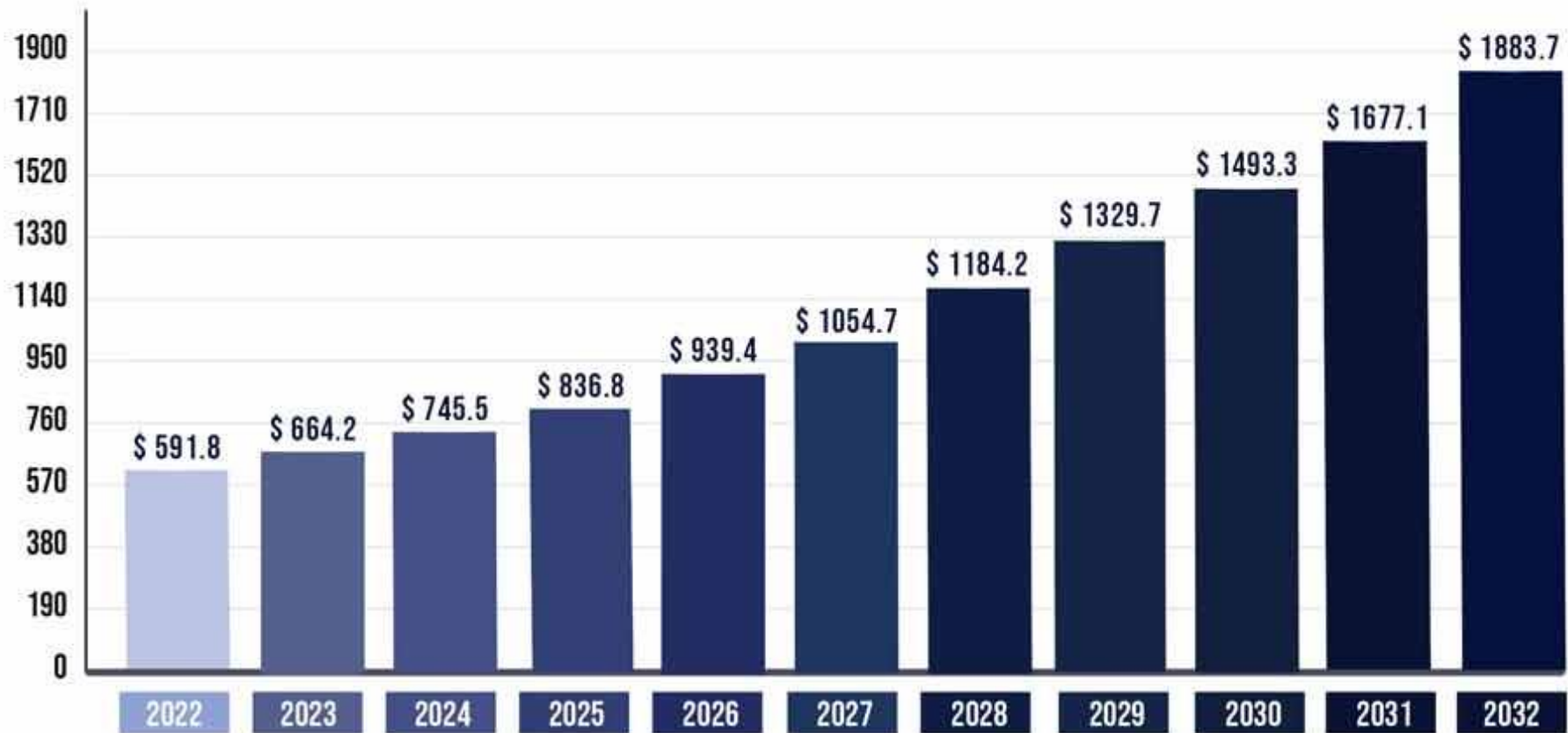
Technology Scaling



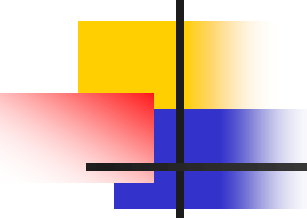
Semiconductor Industry

PRECEDENCE
RESEARCH

SEMICONDUCTOR MARKET SIZE, 2022 TO 2032 (USD BILLION)

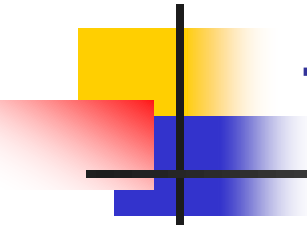


Source: www.precedenceresearch.com



Look into the Future

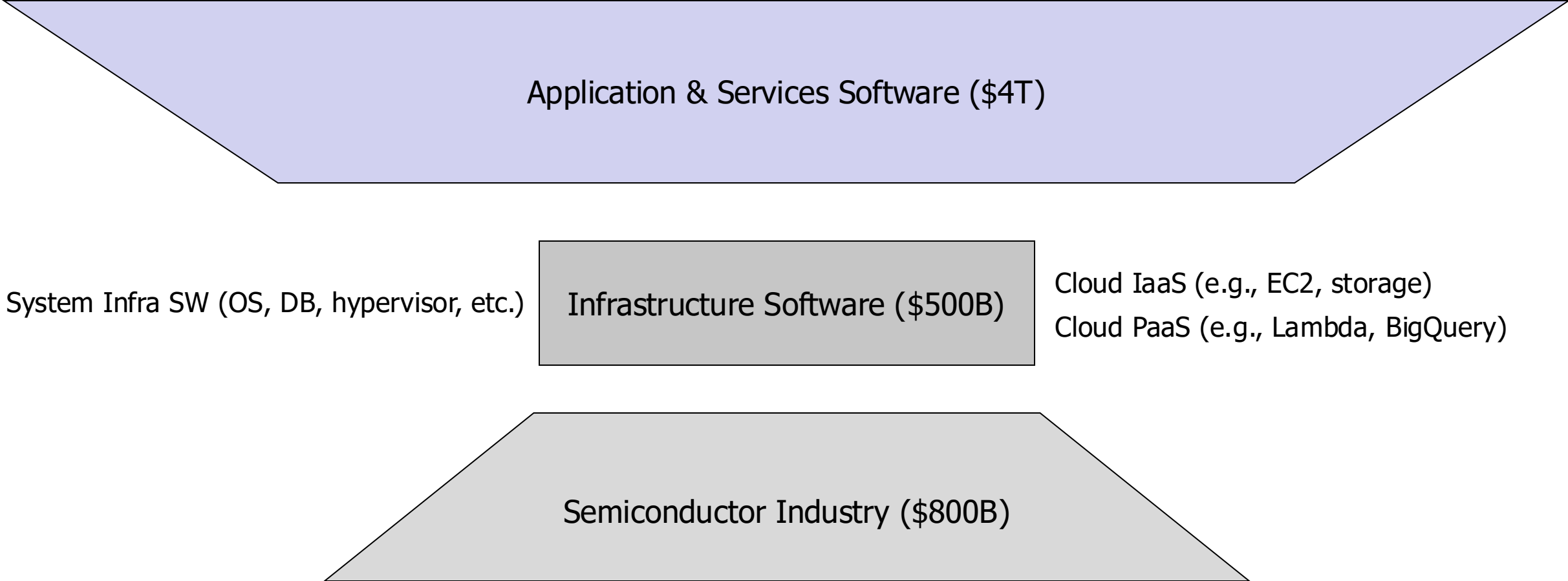
- ❑ Moore's Law as we have known is quickly reaching its end (it does not mean semiconductor technology innovation will stop anytime soon)
- ❑ Personally, I have little faith in any currently known technologies (e.g., quantum computing, spintronics, optical computing, DNA computing, ...) replacing CMOS at a large scale in your career lifetime (hope the future will prove me wrong)
- ❑ The appetite for higher computing power at lower cost/energy will of course never go away



Today's Ecosystem Landscape

How will AI change this landscape?

How will AI change your career perspective?



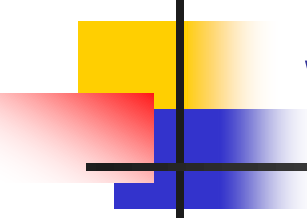
Application & Services Software (\$4T)

System Infra SW (OS, DB, hypervisor, etc.)

Infrastructure Software (\$500B)

Cloud IaaS (e.g., EC2, storage)
Cloud PaaS (e.g., Lambda, BigQuery)

Semiconductor Industry (\$800B)

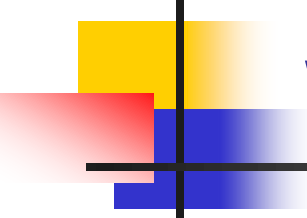


Why & What?

Modern Developers Must Think Beyond Code, Like Infrastructure Architects

- ❑ Traditional software engineering **abstracts the hardware**, but these invisible layers shape:
 - App performance (latency, IOPS, bandwidth)
 - Team productivity (debuggability, tooling, deployment cycles)
 - Business outcomes (cloud spend, carbon footprint, reliability)
- ❑ Understanding infrastructure is now a ***career accelerator***
 - ✓ Infrastructure knowledge multiplies your impact across the stack

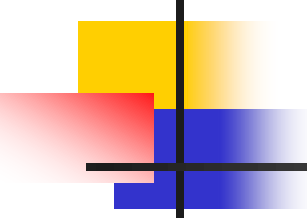
You don't need to be a chip designer. But you do need to understand **what's under the hood**
because **increasingly**, performance, cost, and innovation live there



Why? & What?

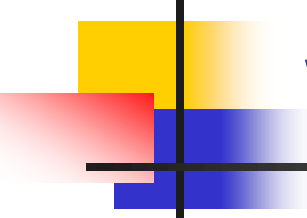
Better understand the computing system infrastructure

- ❑ Know the hardware platform better/deeper
 - ✓ CPU and GPU microarchitecture
 - ✓ Memory and storage hierarchy
 - ✓ Networking and data center architecture
- ❑ Deep dive into the infrastructure software design principles
 - ✓ Two cornerstones: Database and operating system (OS)
 - ✓ Indexing data structures (B-tree, hash table, LSM tree, etc.)
 - ✓ Data atomicity, consistency, isolation, durability



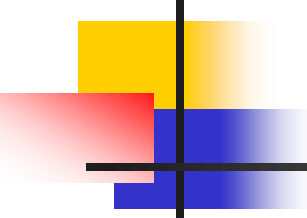
General Information

- ☐ **Why** take this course?
- ☐ **What** is this course all about?
- ☐ **Who** is teaching this course?
- ☐ **How** will this course work?



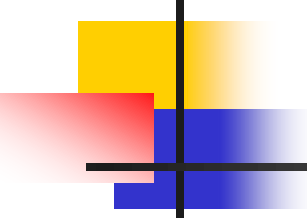
Who?

- ❑ Professor Tong Zhang, 23 years at RPI, IEEE Fellow
 - Email: zhangt4@rpi.edu
 - Office hour: Mondays 1:00-2:00PM, or appointment
 - <https://sites.ecse.rpi.edu/~tzhang/>
- ❑ Current research: Memory-centric computing
- ❑ Work on something practically useful
 - LDPC coding system VLSI architecture design solution being used by all the commercial HDDs and SSDs today
 - Co-founder and Chief Scientist of ScaleFlux (<https://www.scaleflux.com>)
 - ✓ A chip design company aiming to revolutionize memory/storage sub-systems



General Information

- ☐ **Why** take this course?
- ☐ **What** is this course all about?
- ☐ **Who** is teaching this course?
- ☐ **How** will this course work?

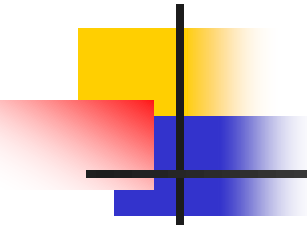


How?

Better understand the computing system infrastructure

- ChatGPT can show you all the facts about computing systems (and pretty much everything else)
- AI rapidly makes “knowledge” so easily accessible/consumable and commoditized
- Will you directly use most knowledge you learn at school in your future career?
- Computer architecture/system are **much more** art than science



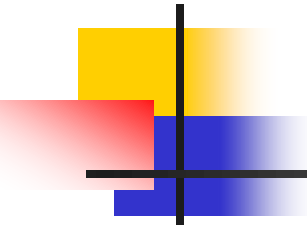


How?

We learn by doing --- through discussions, analysis, and critique across the following threads

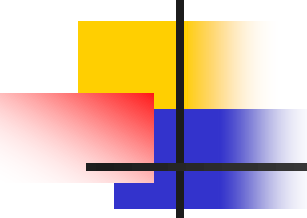
Threads	HW	Infra SW
How to speed up computing?	μarch (pipelines/caches), memory hierarchy (DDR/HBM/SSD), accelerators (GPU/TPU), PCIe/NVLink, networking, ...	vectorization, threading/runtimes, data structures, async I/O & batching, kernel bypassing, RDMA, ...
How to make computing more efficient (e.g., cost, energy)?	heterogeneous computing, SmartNIC/DPU, memory compression, memory pooling, storage-class memory, ...	Data layout, compression, caching, tiering, TCO model, ...
How to keep systems reliable, secure, and correct?	ECC, encryption, confidential computing, RAID, device RAS, ...	checksums & replication/erasure coding, ACID, consensus, ...

- Remember, computer architecture/system is much more art than science
- Goal: become a true problem solver with good instinct, not just be knowledgeable
 - Focus on why & how, not what (ever read/heard of “Thinking, Fast & Slow”?)
 - Learn from failures as much as successes: why some nice ideas didn’t work in practice



Dual-Track Projects

- ❑ Class-wide profiling projects (due by Oct. 1)
 - Gain deeper understanding of CPU, cache/DRAM, and SSD performance
- ❑ Track A: Programming Projects (Well-Defined)
 - Reinforce concepts through measurable implementation & benchmarking
 - Develop the skill of experiment design & result interpretation
- ❑ Track B: Research-Oriented Projects (Exploratory)
 - Develop skills in reading papers, forming hypotheses, designing experiments, and defending conclusions
 - Deliverables include prototype/simulation results, and a conference-style report/talk
- ❑ All the projects are individual projects, of course discussions are encouraged
- ❑ Publish your codes, report and results on Github (show to your future employer)
- ❑ Use of AI programming tools is of course permitted



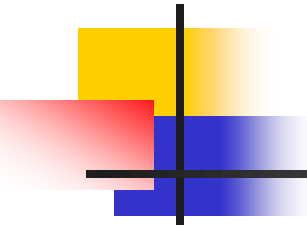
Dual-Track Projects

☐ Track A projects (all)

1. Dense/Sparse Matrix-Matrix Multiplication
2. Vectorized Columnar Compression & Scan Engine
3. Approximate Membership: XOR Filter vs Cuckoo Filter vs Quotient Filter

☐ Track B projects (choose one)

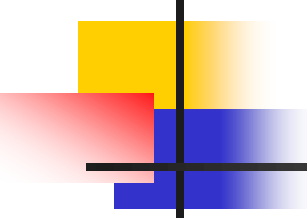
1. Re-think the design of in-memory key-value stores (e.g., tree/hash-based) or algorithms (e.g., sorting, searching) in the presence of HW-based main memory compression
2. Customized programmable core to support future ultra-high IOPS AI-oriented SSD
3. Re-think the design of data structures or algorithms in the presence of GPU + ultra-high IOPS AI-oriented SSD



How? Grading

4000-level	6000-level
☐ 10% participation (in-class, Teams) ☐ 90% course projects	☐ 10% participation (in-class, Teams) ☐ 80% course projects ☐ 10% research report

- ☐ Research report (graduate students only): end of the semester



How?

- ❑ Lecture notes & on-line discussions: Course WebEx Teams space